The CAP Theorem

what it really says, and what it doesn't

Watch the 3-min video

In one line: when the network splits, a distributed system must choose between answering with possibly stale data (availability) and refusing to answer (consistency). Partitions are not optional, so "CA" is not a real choice.

1 Precise definitions

"Consistency, availability, partition tolerance: pick two" is catchy, and misleading. The theorem (Brewer's conjecture, proved by Gilbert and Lynch in 2002) uses each word in a narrow sense:

Property	Meaning in CAP	Not to be confused with
C · Consistency	**Linearizability**: every read returns the latest completed write, as if there were one copy of the data.	The C in ACID (database constraints)
A · Availability	Every request to a non-failed node gets a non-error response, without a time limit or a "try again".	"99.99% uptime" (an SLO)
P · Partition tolerance	The system keeps operating when the network drops or delays messages between nodes.	Surviving a crashed server

2 What happens during a partition

Take two replicas, A and B. The link between them breaks. Client 1 writes `x = 2` to A, which cannot tell B. Client 2 then reads `x` from B, which still has the old value 1.

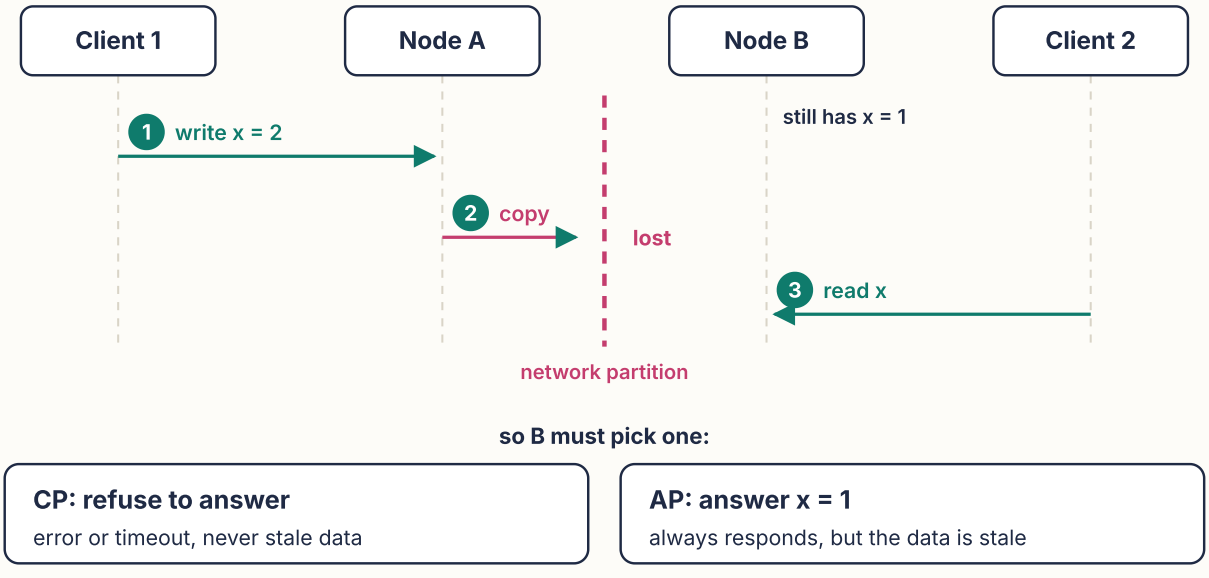


Figure 1. During a partition, node B cannot both stay consistent and keep answering. Whatever it does, it gives up one of the two.

That is the whole theorem: **when a partition happens, you choose consistency or availability.** You cannot opt out of partitions, because real networks lose packets, switches fail and cloud zones disconnect. So for any system spread over a network, "CA" describes either a single machine or a system that pretends the network never breaks.

3 What CAP does not say

- **It says nothing about normal operation.** With no partition, a system can be both consistent and available. CAP only describes the failure window.
- **It is not a permanent label.** The choice can differ per operation, per table or even per query.
- **It ignores latency.** Even without failures, stronger consistency costs round trips between replicas. That everyday trade-off is what PACELC adds.

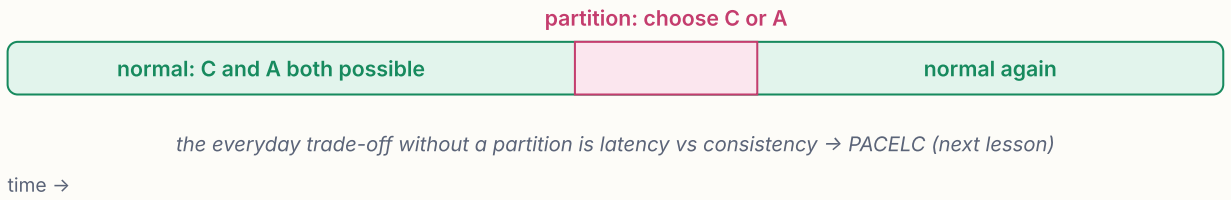


Figure 2. Partitions are short, rare windows. CAP describes behaviour inside them; PACELC adds the latency vs consistency choice for the rest of the time.

4 CP and AP systems in practice

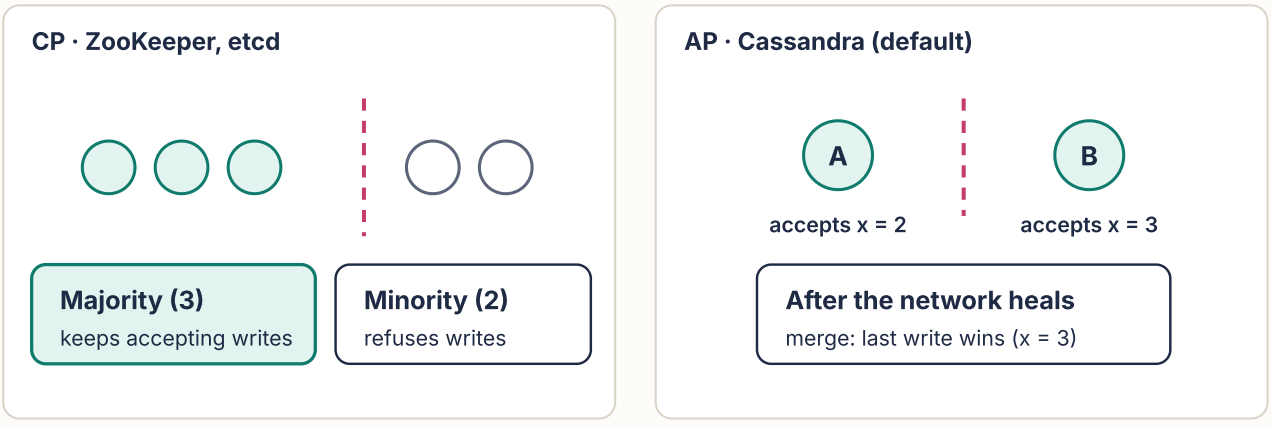


Figure 3. Left: a 5-node CP cluster split 3 / 2; only the majority side accepts writes. Right: an AP store accepts writes on both sides and merges them when the network heals (here, by keeping the later write).

	CP: choose consistency	AP: choose availability
During a partition	The minority side rejects or times out requests	Every reachable node keeps reading and writing
How	Majority quorum (Raft, ZAB): a write commits only on more than half the nodes	Local writes, then repair: hinted handoff, read repair, last-write-wins or CRDTs
Examples	ZooKeeper, etcd, Consul; Spanner (CP with very high availability)	Cassandra and DynamoDB in their default modes, DNS, CDNs
Good for	Leader election, locks, configuration, bank balances, inventory counts	Shopping carts, social feeds, likes, metrics, sessions
Cost	Errors or waiting during failures	Stale reads and write conflicts to resolve

TUNABLE CONSISTENCY (CASSANDRA)

With 3 replicas (N = 3), each query picks how many replicas must answer: `ONE`, `QUORUM` (2 of 3) or `ALL`. If the replicas written (W) plus the replicas read (R) is greater than N, every read overlaps the latest write: `W + R > N`, for example QUORUM writes and QUORUM reads (2 + 2 > 3). `ONE` / `ONE` is faster and more available, but can return stale data. So one database can behave CP-like for one query and AP-like for the next.

HOW TO ANSWER IN AN INTERVIEW

Don't stop at a letter. Describe behaviour: “For the shopping cart I'd stay available during a partition and merge carts afterwards, because a lost sale costs more than a duplicate item. For payments I'd reject writes on the minority side, because a double charge is worse than a short error.”

COMMON MISTAKES

- “We'll pick CA”: impossible across a network.
- Treating CAP consistency as ACID consistency.
- Labelling a whole database CP or AP when its behaviour is tunable per query.
- Forgetting that CAP applies only during a partition.

5 Key terms

Linearizability: reads see the latest write, as if one copy

Quorum: a majority of replicas (more than half)

AP: answer rather than refuse

Partition: nodes up but unable to talk

CP: refuse rather than be wrong

W + R > N: read and write sets overlap → consistent reads

6 Interview questions

Q1. Explain the CAP theorem precisely. BASIC

During a network partition, a distributed system cannot provide both linearizable consistency and total availability: a node cut off from the latest write must either refuse the request or return possibly stale data.

Q2. Why is CA not a real option? BASIC

Partitions are not optional in a distributed system: networks fail. CA only describes a single node, or a system that assumes the network never breaks. The real choice is how to behave when a partition happens.

Q3. How is consistency in CAP different from the C in ACID? BASIC

CAP's C is linearizability: every read sees the latest write across replicas. ACID's C means a transaction keeps the database's rules (constraints, invariants) intact. A single-node database can be ACID-consistent with no replicas at all.

Q4. Which would you choose for a shopping cart, and for a payment ledger? SCENARIO

Cart: AP. Keep accepting adds on both sides and merge afterwards; a duplicate item is cheap, a lost sale is not. Ledger: CP. Only the majority side commits; a short error is better than a double spend.

Q5. You're designing a distributed lock service. CP or AP? SCENARIO

CP. Two clients holding the same lock is a correctness bug. Use a majority quorum (etcd, ZooKeeper) with leases, and give each lock a fencing token so a client whose lease expired can't keep writing. The minority side refuses lock requests during a partition.

Q6. A 5-node etcd cluster splits into 3 and 2. What happens on each side? SCENARIO

The side with 3 has a majority: it keeps or elects a leader and keeps accepting writes. The side with 2 can't reach a quorum, so it rejects writes and linearizable reads. When the network heals, the 2 nodes catch up from the leader's log.

Q7. Is Cassandra AP or CP? TRICKY

AP by default, but tunable per query: with QUORUM reads and writes (W + R > N) it returns the latest acknowledged write, and those queries fail when a quorum can't be reached. Describe the behaviour, not just the letters.

Q8. With N = 3, W = 2 and R = 2, is every read guaranteed to be the latest? TRICKY

The read and write sets always overlap, so a read reaches at least one replica with the latest acknowledged write. But with last-write-wins timestamps, clock skew can make an older concurrent write win, and a failed partial write can show up later. It's strong in practice, not perfectly linearizable.

Next: SD-A-014 · PACELC Theorem Explained backendroadmap.com